

Q&A

Q1. What is the motivation of the X functional?

A1. The starting point is a stronger object than the forward KL, the *detailed balance divergence*

$$\text{DB}(\mu\|\nu) = \int_{\mathbb{R}^d} \mu(x) \left| \nabla \log \frac{\mu(x)}{\nu(x)} \right| dx.$$

Matching ν to μ is the same as flattening the log-ratio $\log(\mu/\nu)$, and the gradient measures exactly this variation, so DB vanishes if and only if $\mu(x)/\nu(x)$ is constant on the support of μ , that is $\nu = \mu$. Where the forward KL constrains only the mean of $\log(\mu/\nu)$ under the target, the gradient in DB forces the whole ratio $\mu(x)/\nu(x)$ to be constant. It is closely related to the Fisher information, obtained by squaring the gradient norm $|\nabla \log(\mu/\nu)|$ in DB.

This choice has one fatal drawback. The gradient $\nabla \log(\mu/\nu)$ involves the score of the pushforward ν , whose evaluation needs the full Jacobian J_G of the flow — not merely its determinant, which the flow already provides — and is therefore extremely expensive for a deep map.

The remedy is to replace the gradient, an infinitesimal difference at a single point, by a finite difference between two points, which yields the first X functional

$$X_\mu(\mu\|\nu) = \iint_{\mathbb{R}^d \times \mathbb{R}^d} \mu(x) \mu(y) \left| \log \frac{\mu(x)}{\nu(x)} - \log \frac{\mu(y)}{\nu(y)} \right| dx dy.$$

It keeps the property that motivated DB — it vanishes exactly when $\log(\mu/\nu)$ is constant on the support of μ — but removes the gradient: it reads off only the log-ratio $\log(\mu/\nu)$, which every normalizing flow provides cheaply, so the expensive Jacobian never appears. The pairwise difference also cancels the unknown normalizer of μ , which enters the log-ratio as an additive constant and drops out of the difference.

Q2. Why is the X regularization defined with an absolute value rather than a square?

A2. We did consider the squared variant — the pairwise difference under a square $|\cdot|^2$ in place of the absolute value $|\cdot|$ in X_μ — and tested both on the 2D benchmarks. The absolute value was consistently the better of the two, and it almost never caused the numerical singularities the square is prone to. We suspect the reason is that the absolute value keeps X_μ closest to the forward KL, on the same linear scale in the log-ratio, so the two can share a single learning rate; the square grows quadratically and sits on a different scale.

Q3. Can X_μ be used on its own, without the forward KL, as the loss?

A3. No. On the 2D benchmarks, X_μ alone does not even make the flow converge. We conjecture why: the forward KL, driven by Jensen’s inequality, shapes the flow globally, whereas X_μ only adjusts the log-ratio pointwise and locally, and a purely local signal cannot drive the flow to converge. The forward KL is therefore indispensable, and its weight against X_μ is a genuine trade-off between accuracy and cost; we recommend an equal 1:1 weight as the default.

Q4. Is the X-regularized forward KL always better, and how does one choose between it and the plain forward KL in practice?

A4. Not always. In most of the small-batch cases we tested, the X-regularized losses were better; at large batch, though, the gap between the forward KL and its X-regularized versions shrinks. The rule of thumb is simple: when the batch is small, use the X regularization. And if a multi-GPU machine is at hand, there is no need to commit to one loss — train several in parallel within a single stage (forward KL, FAB, KL+X+X, or even FAB+X+X), compare them, and carry the best one forward.

Q5. Where does the mode discovery ability of the X functionals come from, and how does it differ from PT or FAB?

A5. It comes from the mixture functional $X_{(\hat{\mu}+\bar{\nu})/2}$, built on the wide-coverage measure $\hat{\mu}$ that quench and temper (QT) prepares before training. PT is a classical Monte Carlo method that leans on frequent temperature swaps for ergodicity, so it can only move mass between modes a chain already reaches; FAB, and other methods that merely change the divergence, inherit the same limit, since their annealing chains must still cross the barriers and none hunts for a lost mode on its own. QT does find lost modes proactively, off any chain — but its samples are coarse, and feeding them into training directly would bias the target and cost accuracy. The X functional is what makes them usable: it vanishes whenever the log-ratio is constant on the support of its weight, whatever the shape of that weight, so $\hat{\mu}$ only tells the flow where to place mass, never what density to fit. The mixture term thus turns the QT data into a clean training signal, pulling the flow onto the discovered modes while leaving the minimizer $\nu = \mu$ untouched.

Q6. What is the meaning of the Fisher–Rao gradient flow analysis, and does it connect to the real flow training?

A6. Frankly, not much. The Fisher–Rao flow is a continuous-time idealization on the space of densities, whereas real training takes discrete Adam steps on finite batches; we claim no faster transient from it. Its real value, in our view, is the asymptotic accuracy result: under biased annealed samples, adding X_μ provably lowers the accuracy floor at which the bare forward KL is stuck. And since X_μ reuses the forward KL’s own samples and forward pass at negligible cost, it is about as close to a free lunch as one finds — a cheap term that simply buys accuracy.